

PyTorch and Keras, side by side

Three columns on the screen: your NumPy, PyTorch's explicit loop, Keras's compile and fit. Start all three from the same thirteen hundred numbers, feed them the same batches in the same order, and the three loss curves land on top of each other. Then one line — `loss.backward()` — replaces the function you derived by hand two weeks ago.

Unit 3 · Libraries

80 minutes

Lab 12, part 2

Colab or atd_env

Two philosophies

Three columns

Identical curves

Autograd

Lab 12 · part 2

Before Tuesday

PART 1 · 10 MINUTES

Two frameworks, two philosophies

1



PyTorch

You write the loop. Tensors, a model made of layers, a loss, an optimiser — and four lines per step that look exactly like yours. Dominant in research.



Keras

You describe the model and call `fit`. The loop exists, but inside the library. Runs on TensorFlow (and other back ends) and is popular in production and teaching.

Neither is better in general. PyTorch shows you the loop; Keras hides it. Having written the loop yourself, you can choose deliberately — and read either one.

Three columns

The same 64–16–1 network, three times.

2

Step	Your NumPy	PyTorch	Keras
Model	<code>init_params</code> + <code>forward</code>	<code>nn.Sequential(nn.Linear(64,16), nn.Sigmoid(), nn.Linear(16,1), nn.Sigmoid())</code>	<code>Sequential([Dense(16, "sigmoid"), Dense(1, "sigmoid")])</code>
Loss	<code>bce</code>	<code>nn.BCELoss()</code>	<code>loss="binary_crossentropy"</code>
Gradients	<code>backward(p, X, y)</code>	<code>loss.backward()</code>	inside <code>train_on_batch</code> / <code>fit</code>
Update	<code>p[k] -= lr * g[k]</code>	<code>opt.step()</code>	inside, from <code>SGD(learning_rate=...)</code>
Reset	— (fresh each call)	<code>opt.zero_grad()</code>	inside

the PyTorch column, from Lab 12 part 2

PYTHON

```

for idx in batch_order:
    for s in range(0, len(X_train), 32):
        b = torch.from_numpy(idx[s:s + 32])
        opt.zero_grad()
        loss = loss_fn(model(Xt[b]), yt[b])
        loss.backward()
        opt.step()

```

the same shuffles as NumPy

forget last step's gradients

forward + loss

every gradient – your session 14

every update – your session 9



The zero_grad trap

PyTorch *adds* new gradients to old ones unless told to forget. Leave out `opt.zero_grad()` and each step uses the sum of every gradient so far. It runs; it trains badly; nothing warns you.

Making three frameworks agree exactly

3

Tuesday's gap came from what we did not control. Today we control everything.

- 1 **Same starting weights.** Create them once in NumPy and copy them in. PyTorch stores a layer's weight as *(out, in)*, so it receives `W1.T`. Keras stores *(in, out)* — your orientation — so it receives `W1` unchanged.
- 2 **Same batches.** Draw the shuffles once, as a list of permutations, and let all three loops walk that list.
- 3 **Same arithmetic.** float64 everywhere, the same loss, plain SGD with the same learning rate.

WHAT TO EXPECT

NumPy and PyTorch agree to around 10^{-12} — identical arithmetic in a different order. Keras lands within about 10^{-6} , because its cross-entropy clips probabilities at 10^{-7} where yours adds 10^{-12} . **A gap of 0.01 means a blank is wrong**, most often a missing transpose.

Autograd: why `loss.backward()` lands

4

As PyTorch runs the forward pass, it records every operation in a graph: this matrix multiply fed that addition fed that sigmoid. `loss.backward()` walks the graph in reverse and applies the chain rule at each node — exactly the product-along-the-path from session 14, for every parameter, automatically. This is **automatic differentiation**, and it is why nobody derives backward passes for large models by hand.

WHY IT DOES NOT FEEL LIKE MAGIC TODAY

You have derived the same gradients on the board, coded them in four lines, and proved them with a numerical check. `loss.backward()` is the function you wrote, generalised. A student who started with PyTorch has to trust it; you can **check it** — and in session 19 that ability is what finds the bugs.

Three questions

Q1. Which framework needs the transpose, and why? PyTorch: `nn.Linear` computes $x \cdot W^T + b$ with W stored as (out, in). Our convention is $X @ W$ with W as (in, out).

Q2. What does `opt.step()` do that your code did in one line? `p[k] -= lr * g[k]` for every parameter the optimiser was given.

Q3. Keras's curve is 10^{-6} away from NumPy's. Is Keras wrong? No — its loss clips probabilities differently. The gap is explained, tiny, and stable. An unexplained gap of any size is what deserves suspicion.

LAB 12, PART 2 · 20 MINUTES IN CLASS

Three frameworks, one curve

5

Lab 12 — `CSC4315_Lab12_One_Model_Three_Frameworks`, Part 2. Three blanks in PyTorch, two in Keras, then one plot with three curves on it and two printed gaps.

Before Tuesday

1. **Finish Lab 12.**
2. **Try the extension:** swap SGD for Adam in PyTorch and plot both curves. Tuesday explains what you saw.



Session 19 — Tuesday 10 November, 16:00

What you gain, and when training fails: optimisers, initialisation and the GPU — then a network that will not learn, diagnosed live.